

About this document

This is the production-corrected version of the CoolDesk PostgreSQL schema. Every issue identified in the v1.0 architectural review has been resolved. A full improvements summary appears in Section F.

Design decisions and assumptions are called out inline throughout the document.

PostgreSQL 14+ | NestJS Modular Monolith | Version 2.0

A. Entity Breakdown

The schema is organised into 23 tables (22 original + the previously missing `scheduling_conflicts` table) across six logical groups. All entities have explicit referential integrity, cascade policies, and constraints.

Table	Purpose
<code>virtual_customers</code>	Customer identity anchor. Holds the VCID (UUID PK). No PII stored here.
<code>customer_phones</code>	All phone numbers linked to a VCID. Many phones per VCID.
<code>users</code>	Owner, Office Staff, Technician. Single table with role enum.
<code>dealers</code>	Separate entity. Has own business details, region, and brand links.
<code>dealer_credentials</code>	Auth credentials for dealers, separated from the business entity. NEW in v2.
<code>dealer_brands</code>	M:N join between dealers and brands.
<code>brands</code>	AC brands. Activatable/deactivatable. Colour-coded.
<code>jobs</code>	Central entity. Covers both installations and complaints. type + status drive lifecycle.
<code>job_units</code>	Child rows for multi-unit jobs. One row per AC unit.
<code>job_assignments</code>	Assignment history per job. One active row; superseded rows retained.
<code>job_cancellation_requests</code>	Dealer-submitted cancellation requests, extracted from jobs row. NEW in v2.
<code>revisits</code>	Child revisit records linked to a parent complaint job.
<code>revisit_assignments</code>	Per-revisit technician assignment history.
<code>payments</code>	1:1 with job. Records amount, method, status, and edit trail.
<code>payment_methods</code>	Owner-managed list of accepted payment methods.
<code>job_timeline</code>	Append-only audit log for every significant job event.
<code>notifications</code>	In-app notification store with deduplication.
<code>scheduling_conflicts</code>	Log of acknowledged scheduling conflicts. Present in v1 entity list but DDL was missing.

Table	Purpose
analytics_technician_daily	Precomputed daily rollup per technician.
analytics_business_daily	Precomputed daily rollup for business-wide metrics.
analytics_brand_daily	Precomputed daily rollup per brand.
analytics_dealer_daily	Precomputed daily rollup per dealer.
system_config	Key-value store for owner-managed operational thresholds.
customer_reviews	Optional Module 19. Post-completion review links and ratings.

B. Full Database Schema (DDL)

All DDL is standard PostgreSQL 14+. Run sections in order shown. Enum types must be created before the tables that reference them.

B.1 — Enums

```
-- =====
-- ENUMS
-- =====

CREATE TYPE user_role AS ENUM (
    'owner',
    'office_staff',
    'technician'
);

CREATE TYPE job_type AS ENUM (
    'installation',
    'complaint'
);

CREATE TYPE job_source AS ENUM (
    'direct',
    'via_dealer'
);

-- Combined job status enum (v1 stored this as TEXT — now a proper enum).
-- The validate_job_status trigger enforces per-type validity.
CREATE TYPE job_status AS ENUM (
    'pending_schedule',
    'scheduled',
    'assigned',
    'acknowledged',
    'in_transit',
    'in_process',
    'completed',
    'new',
    'resolved',
    'needs_revisit',
    'revisit_scheduled',
    'resolved_on_revisit',

```

```

    'cancellation_requested',
    'cancelled'
);

CREATE TYPE payment_status AS ENUM (
    'collected',
    'pending'
);

-- Renamed from visit_outcome to avoid collision with the
-- column of the same name on jobs and revisits.
CREATE TYPE arrival_outcome AS ENUM (
    'on_time',
    'late',
    'no_show',
    'rescheduled'
);

CREATE TYPE revisit_reason AS ENUM (
    'part_unavailable',
    'customer_not_home',
    'issue_recurring',
    'further_diagnosis_required',
    'custom'
);

-- Enum for revisit resolution (replaces TEXT CHECK in v1).
CREATE TYPE revisit_outcome AS ENUM (
    'resolved',
    'needs_revisit'
);

CREATE TYPE cancellation_request_status AS ENUM (
    'pending',
    'approved',
    'rejected'
);

CREATE TYPE timeline_event_type AS ENUM (
    'status_transition', 'status_undo', 'status_rollback',
    'assignment', 'reassignment', 'scheduling_conflict_ack',
    'payment_recorded', 'payment_edited', 'payment_void',
    'payment_retained', 'cancellation', 'cancellation_request',
    'cancellation_approved', 'cancellation_rejected', 'reopened',
    'customer_linked', 'customer_link_dismissed', 'note_added',
    'revisit_created', 'revisit_scheduled', 'rollback_payment_voided',
    'system_event' -- for background-worker entries with no human actor
);

CREATE TYPE notification_event AS ENUM (
    'dealer_job_submitted', 'job_assigned',
    'cancellation_request_submitted', 'cancellation_request_outcome',
    'job_unacknowledged', 'no_show_flagged',
    'repeat_complaint_detected', 'frequent_complaint_detected',
    'third_revisit_reached'
);

CREATE TYPE review_mode AS ENUM (
    'off', 'optional', 'mandatory'
);

```

```
|);
```

B.2 — Customer Identity

```
-- -----
-- VIRTUAL CUSTOMER IDENTITY
-- -----

CREATE TABLE virtual_customers (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  is_frequent BOOLEAN       NOT NULL DEFAULT FALSE,
  frequent_flagged_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE customer_phones (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  vcid        UUID          NOT NULL
    REFERENCES virtual_customers(id) ON DELETE RESTRICT,
  phone       TEXT          NOT NULL,
  is_primary  BOOLEAN       NOT NULL DEFAULT FALSE,
  linked_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- FK enforced (was a bare comment in v1)
  linked_by   UUID          REFERENCES users(id) ON DELETE SET NULL,
  UNIQUE (vcid, phone)
);

CREATE INDEX idx_customer_phones_phone ON customer_phones (phone);
CREATE INDEX idx_customer_phones_vcid  ON customer_phones (vcid);
```

B.3 — Users, Dealers & Brands

```
-- -----
-- USERS
-- -----

CREATE TABLE users (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  role        user_role     NOT NULL,
  full_name   TEXT          NOT NULL,
  email       TEXT          UNIQUE NOT NULL,
  password_hash TEXT        NOT NULL,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  is_deleted  BOOLEAN       NOT NULL DEFAULT FALSE, -- consistent soft-delete
  created_by  UUID          REFERENCES users(id) ON DELETE SET NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- -----
-- DEALERS
-- -----

-- Auth credentials extracted to dealer_credentials (see below).
```

```

CREATE TABLE dealers (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  business_name TEXT        NOT NULL,
  contact_name TEXT        NOT NULL,
  email       TEXT        UNIQUE NOT NULL,
  region      TEXT,
  is_active   BOOLEAN      NOT NULL DEFAULT TRUE,
  is_deleted  BOOLEAN      NOT NULL DEFAULT FALSE,
  created_by  UUID          NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- Auth credentials separated from the dealer business entity.
-- Allows dealers without portal access, and future OAuth/SSO addition
-- without altering the dealers table.
CREATE TABLE dealer_credentials (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  dealer_id   UUID          NOT NULL UNIQUE
                        REFERENCES dealers(id) ON DELETE CASCADE,
  password_hash TEXT        NOT NULL,
  last_login_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- -----
-- BRANDS
-- -----

CREATE TABLE brands (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT        NOT NULL UNIQUE,
  colour_hex  CHAR(6),
  is_active   BOOLEAN      NOT NULL DEFAULT TRUE,
  is_deleted  BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE dealer_brands (
  dealer_id  UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
  brand_id   UUID NOT NULL REFERENCES brands(id) ON DELETE RESTRICT,
  PRIMARY KEY (dealer_id, brand_id)
);

```

B.4 — Jobs (Core Entity)

```

-- -----
-- JOBS
-- -----
-- Cancellation request columns extracted to job_cancellation_requests (B.4a).

CREATE TABLE jobs (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  -- Classification

```

```

type          job_type    NOT NULL,
source        job_source  NOT NULL,
-- status is now a proper enum (was TEXT in v1).
-- The validate_job_status trigger enforces per-type validity.
status        job_status  NOT NULL,

-- Brand (mandatory)
brand_id      UUID        NOT NULL REFERENCES brands(id) ON DELETE RESTRICT,

-- Dealer link (required when source = via_dealer)
dealer_id     UUID        REFERENCES dealers(id) ON DELETE RESTRICT,

-- Customer identity
vcid         UUID        NOT NULL REFERENCES virtual_customers(id) ON DELETE
RESTRICT,

-- Embedded PII (see Assumption A-1)
customer_name TEXT        NOT NULL,
address       TEXT        NOT NULL,

-- Flags derived from VCID history.
-- is_repeat and is_frequent store context columns so that
-- retroactive threshold changes can be audited (see Assumption A-2).
is_repeat     BOOLEAN NOT NULL DEFAULT FALSE,
repeat_window_days_used INT,      -- threshold in effect when flag was set
is_frequent   BOOLEAN NOT NULL DEFAULT FALSE,
frequent_threshold_used INT,      -- threshold in effect when flag was set
frequent_window_used INT,
-- Set automatically by trg_chronic (trigger on revisits, B.6)
is_chronic    BOOLEAN NOT NULL DEFAULT FALSE,

-- Scheduling
scheduled_at   TIMESTAMPTZ,
actual_arrival TIMESTAMPTZ,
-- Renamed enum: was visit_outcome, now arrival_outcome
visit_outcome  arrival_outcome,
late_by_minutes INT CHECK (late_by_minutes >= 0),

-- Complaint-specific (NULL on installations)
issue_description TEXT,

-- Cancellation/closure (direct cancel only; requests in
job_cancellation_requests)
cancelled_at   TIMESTAMPTZ,
cancelled_by   UUID REFERENCES users(id) ON DELETE SET NULL,
cancellation_reason TEXT,

-- Optimistic locking
version       INT          NOT NULL DEFAULT 0,
created_by    UUID          REFERENCES users(id) ON DELETE SET NULL,
created_at    TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
updated_at    TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
is_deleted    BOOLEAN       NOT NULL DEFAULT FALSE,

-- Data integrity rules
CONSTRAINT chk_dealer_source
    CHECK (source <> 'via_dealer' OR dealer_id IS NOT NULL),

CONSTRAINT chk_cancel_meta

```

```

        CHECK (
            cancelled_at IS NULL OR
            (cancelled_by IS NOT NULL AND cancellation_reason IS NOT NULL)
        ),

        CONSTRAINT chk_cancel_order
            CHECK (cancelled_at >= created_at OR cancelled_at IS NULL)
    );

-- — All 6 indexes described in v1 Section D are now created —
CREATE INDEX idx_jobs_vcid_created
    ON jobs (vcid, created_at DESC)
    WHERE is_deleted = FALSE;

CREATE INDEX idx_jobs_status_type
    ON jobs (status, type)
    WHERE is_deleted = FALSE;

CREATE INDEX idx_jobs_dealer
    ON jobs (dealer_id, created_at DESC)
    WHERE is_deleted = FALSE;

CREATE INDEX idx_jobs_brand      ON jobs (brand_id);
CREATE INDEX idx_jobs_scheduled ON jobs (scheduled_at);
CREATE INDEX idx_jobs_is_chronic ON jobs (is_chronic)
    WHERE is_chronic = TRUE;

CREATE INDEX idx_jobs_not_deleted
    ON jobs (id) WHERE is_deleted = FALSE;

```

B.4a — Job Cancellation Requests (extracted from jobs row)

```

-- Dealer-submitted cancellation requests extracted from the jobs row.
-- Removes 6 nullable columns from the hot jobs table and enforces
-- temporal ordering at the DB level.

CREATE TABLE job_cancellation_requests (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id      UUID          NOT NULL UNIQUE
        REFERENCES jobs(id) ON DELETE RESTRICT,
    requested_by_dealer_id UUID NOT NULL
        REFERENCES dealers(id) ON DELETE RESTRICT,
    reason      TEXT          NOT NULL,
    requested_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    status      cancellation_request_status NOT NULL DEFAULT 'pending',
    decided_by  UUID          REFERENCES users(id) ON DELETE SET NULL,
    decided_at  TIMESTAMPTZ,
    decision_note TEXT,

    CONSTRAINT chk_decision_meta
        CHECK (
            (status = 'pending' AND decided_by IS NULL AND decided_at IS NULL) OR
            (status <> 'pending' AND decided_by IS NOT NULL AND decided_at IS NOT
NULL)
        ),

    CONSTRAINT chk_decided_order
        CHECK (decided_at IS NULL OR decided_at >= requested_at)

```

```
);

CREATE INDEX idx_cancel_req_job ON job_cancellation_requests (job_id);
```

B.4b — Job Units

```
CREATE TABLE job_units (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL
                  REFERENCES jobs(id) ON DELETE CASCADE,
  model       TEXT          NOT NULL,
  unit_type   TEXT          NOT NULL,
  num_units   SMALLINT CHECK (num_units > 0),
  fault_description TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_job_units_job ON job_units (job_id);
```

B.5 — Assignments

```
-- -----
-- JOB ASSIGNMENTS
-- -----

CREATE TABLE job_assignments (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL
                  REFERENCES jobs(id) ON DELETE RESTRICT,
  technician_id UUID        NOT NULL
                  REFERENCES users(id) ON DELETE RESTRICT,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  assigned_by  UUID          NOT NULL
                  REFERENCES users(id) ON DELETE RESTRICT,
  assigned_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  acknowledged_at TIMESTAMPTZ,
  unassigned_at TIMESTAMPTZ
);

-- Enforces exactly one active assignment per job.
CREATE UNIQUE INDEX idx_job_assignments_active
  ON job_assignments (job_id)
  WHERE is_active = TRUE;

CREATE INDEX idx_job_assignments_technician ON job_assignments (technician_id);
CREATE INDEX idx_job_assignments_job       ON job_assignments (job_id);
```

B.6 — Revisits

```
CREATE TABLE revisits (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL
                  REFERENCES jobs(id) ON DELETE RESTRICT,
  sequence_number SMALLINT NOT NULL,
```



```

reason          revisit_reason NOT NULL,
custom_reason   TEXT,
scheduled_at    TIMESTAMPTZ,
actual_arrival  TIMESTAMPTZ,
-- Renamed enum: was visit_outcome, now arrival_outcome
visit_outcome   arrival_outcome,
late_by_minutes INT CHECK (late_by_minutes >= 0),
-- Replaced TEXT CHECK with a proper enum (was 'resolved' || 'needs_revisit')
outcome         revisit_outcome,
notes           TEXT,
created_at      TIMESTAMPTZ      NOT NULL DEFAULT NOW(),
updated_at      TIMESTAMPTZ      NOT NULL DEFAULT NOW(),

CONSTRAINT chk_custom_reason
    CHECK (reason <> 'custom' OR custom_reason IS NOT NULL),

    UNIQUE (job_id, sequence_number)
);

CREATE INDEX idx_revisits_job ON revisits (job_id);

-- Trigger: automatically sets jobs.is_chronic = TRUE when
-- sequence_number reaches 3 on a given job.
CREATE OR REPLACE FUNCTION fn_check_chronic()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.sequence_number >= 3 THEN
        UPDATE jobs
        SET     is_chronic = TRUE,
              updated_at = NOW()
        WHERE  id = NEW.job_id
              AND is_chronic = FALSE; -- no-op if already set
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_chronic
AFTER INSERT ON revisits
FOR EACH ROW EXECUTE FUNCTION fn_check_chronic();

-- Revisit assignments (same pattern as job_assignments)
CREATE TABLE revisit_assignments (
    id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    revisit_id        UUID          NOT NULL
                                REFERENCES revisits(id) ON DELETE RESTRICT,
    technician_id     UUID          NOT NULL
                                REFERENCES users(id) ON DELETE RESTRICT,
    is_active         BOOLEAN       NOT NULL DEFAULT TRUE,
    assigned_by       UUID          NOT NULL
                                REFERENCES users(id) ON DELETE RESTRICT,
    assigned_at       TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    acknowledged_at   TIMESTAMPTZ,
    unassigned_at     TIMESTAMPTZ
);

CREATE UNIQUE INDEX idx_revisit_assignments_active
ON revisit_assignments (revisit_id)
WHERE is_active = TRUE;

```

B.7 — Payments

```
CREATE TABLE payment_methods (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        TEXT          NOT NULL UNIQUE,  
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,  
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE payments (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  job_id      UUID          NOT NULL UNIQUE  
              REFERENCES jobs(id) ON DELETE RESTRICT,  
  amount      NUMERIC(12,2) NOT NULL CHECK (amount >= 0),  
  payment_method_id UUID    NOT NULL  
              REFERENCES payment_methods(id) ON DELETE RESTRICT,  
  status      payment_status NOT NULL DEFAULT 'pending',  
  recorded_by UUID          NOT NULL  
              REFERENCES users(id) ON DELETE RESTRICT,  
  recorded_at TIMESTAMPTZ   NOT NULL DEFAULT NOW(),  
  
  -- Tracks who last edited the payment (office staff corrections).  
  -- Absent in v1; added to close the audit gap.  
  last_edited_by  UUID          REFERENCES users(id) ON DELETE SET NULL,  
  last_edited_at  TIMESTAMPTZ,  
  
  -- Void fields (populated when owner voids payment)  
  voided_at      TIMESTAMPTZ,  
  voided_by      UUID REFERENCES users(id) ON DELETE SET NULL,  
  void_reason     TEXT,  
  
  -- Retain fields  
  retained_note   TEXT,  
  
  -- Optimistic locking: version guard is enforced in the service layer.  
  -- See Section H for the UPDATE pattern.  
  version         INT          NOT NULL DEFAULT 0,  
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT chk_void_meta  
    CHECK (  
      voided_at IS NULL OR  
      (voided_by IS NOT NULL AND void_reason IS NOT NULL)  
    ),  
  
  CONSTRAINT chk_edit_meta  
    CHECK (  
      last_edited_at IS NULL OR last_edited_by IS NOT NULL  
    )  
);  
  
CREATE INDEX idx_payments_job ON payments (job_id);
```

B.8 — Timeline (Audit Log)

```
-- Append-only. Rows are never updated or deleted.
-- actor_user_id or actor_dealer_id must be set,
-- OR the event_type must be 'system_event' (background worker).

CREATE TABLE job_timeline (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id            UUID                NOT NULL
                        REFERENCES jobs(id) ON DELETE RESTRICT,
  event_type        timeline_event_type NOT NULL,
  actor_user_id     UUID                REFERENCES users(id)   ON DELETE SET NULL,
  actor_dealer_id   UUID                REFERENCES dealers(id) ON DELETE SET NULL,
  previous_value    JSONB,
  new_value         JSONB,
  reason            TEXT,
  related_job_id    UUID                REFERENCES jobs(id) ON DELETE SET NULL,
  revisit_id        UUID                REFERENCES revisits(id) ON DELETE SET NULL,
  note_text         TEXT,
  occurred_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  -- Every event must have a human actor OR be explicitly system-generated.
  CONSTRAINT chk_actor
    CHECK (
      actor_user_id IS NOT NULL OR
      actor_dealer_id IS NOT NULL OR
      event_type = 'system_event'
    )
  -- No updates ever. Rows are immutable once inserted.
);

CREATE INDEX idx_timeline_job      ON job_timeline (job_id, occurred_at DESC);
CREATE INDEX idx_timeline_event_type ON job_timeline (event_type);
CREATE INDEX idx_timeline_actor_user ON job_timeline (actor_user_id);
```

B.9 — Notifications

```
CREATE TABLE notifications (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type        notification_event NOT NULL,
  job_id            UUID                REFERENCES jobs(id) ON DELETE CASCADE,
  recipient_user_id UUID                REFERENCES users(id) ON DELETE CASCADE,
  recipient_dealer_id UUID              REFERENCES dealers(id) ON DELETE CASCADE,
  is_read           BOOLEAN             NOT NULL DEFAULT FALSE,
  read_at          TIMESTAMPTZ,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT chk_recipient
    CHECK (
      recipient_user_id IS NOT NULL OR
      recipient_dealer_id IS NOT NULL
    )
);

-- Deduplication indexes prevent duplicate notifications from retried workers.
CREATE UNIQUE INDEX idx_notif_dedup_user
  ON notifications (event_type, job_id, recipient_user_id)
```

```

WHERE recipient_user_id IS NOT NULL;

CREATE UNIQUE INDEX idx_notif_dedup_dealer
ON notifications (event_type, job_id, recipient_dealer_id)
WHERE recipient_dealer_id IS NOT NULL;

CREATE INDEX idx_notifications_recipient_user
ON notifications (recipient_user_id, is_read, created_at DESC);
CREATE INDEX idx_notifications_recipient_dealer
ON notifications (recipient_dealer_id, is_read, created_at DESC);
CREATE INDEX idx_notifications_job ON notifications (job_id);

```

B.10 — Scheduling Conflicts (previously missing DDL)

Fix applied

scheduling_conflicts was listed as table #17 in the v1 entity breakdown but had no CREATE TABLE statement anywhere in the DDL. Any write attempt would have caused a runtime error. The table is now fully defined below.

```

CREATE TABLE scheduling_conflicts (
  id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id            UUID          NOT NULL
    REFERENCES jobs(id) ON DELETE RESTRICT,
  conflicting_job_id UUID          NOT NULL
    REFERENCES jobs(id) ON DELETE RESTRICT,
  technician_id     UUID          NOT NULL
    REFERENCES users(id) ON DELETE RESTRICT,
  acknowledged_by   UUID          NOT NULL
    REFERENCES users(id) ON DELETE RESTRICT,
  acknowledged_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  note              TEXT,

  -- A conflict pair is directional; prevent exact duplicates.
  CONSTRAINT chk_not_self_conflict
    CHECK (job_id <> conflicting_job_id),

  UNIQUE (job_id, conflicting_job_id, technician_id)
);

CREATE INDEX idx_conflicts_job ON scheduling_conflicts (job_id);

```

B.11 — System Config

```

CREATE TABLE system_config (
  key      TEXT          PRIMARY KEY,
  value     TEXT          NOT NULL,
  updated_by UUID        NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Seed defaults.
-- v1 used a literal '<owner-uuid>' placeholder which causes a

```

```

-- UUID cast error at runtime. The seed is now done via a migration
-- function that reads the owner's actual id.
CREATE OR REPLACE FUNCTION seed_system_config(p_owner_id UUID)
RETURNS VOID LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO system_config (key, value, updated_by) VALUES
        ('repeat_complaint_window_days', '30', p_owner_id),
        ('frequent_complaint_threshold', '3', p_owner_id),
        ('frequent_complaint_window_days', '90', p_owner_id),
        ('punctuality_grace_period_mins', '15', p_owner_id),
        ('customer_review_mode', 'off', p_owner_id),
        ('undo_window_seconds', '60', p_owner_id)
    ON CONFLICT (key) DO NOTHING;
END;
$$;
-- Call after the owner user row is inserted:
-- SELECT seed_system_config('<actual-owner-uuid>');

```

B.12 — Analytics (Precomputed)

```

-- Dashboard queries hit these tables, not jobs directly.
-- Updated by NestJS workers after every job mutation.
-- See Section I for idempotent UPSERT pattern.

CREATE TABLE analytics_technician_daily (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    technician_id     UUID                NOT NULL
        REFERENCES users(id) ON DELETE RESTRICT,
    date              DATE                NOT NULL,
    jobs_assigned     INT                 NOT NULL DEFAULT 0,
    installations_completed INT           NOT NULL DEFAULT 0,
    complaints_resolved INT              NOT NULL DEFAULT 0,
    revisit_count     INT                 NOT NULL DEFAULT 0,
    no_show_count     INT                 NOT NULL DEFAULT 0,
    on_time_count     INT                 NOT NULL DEFAULT 0,
    late_count        INT                 NOT NULL DEFAULT 0,
    total_payment_collected NUMERIC(14,2) NOT NULL DEFAULT 0,
    pending_payment_total NUMERIC(14,2) NOT NULL DEFAULT 0,
    avg_resolution_mins NUMERIC(10,2),
    avg_acknowledgment_mins NUMERIC(10,2),
    first_visit_resolved INT              NOT NULL DEFAULT 0,
    -- Module 19 (NULL when feature is off; controlled by system_config)
    avg_star_rating    NUMERIC(3,2),
    updated_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
    UNIQUE (technician_id, date)
);

-- Trigger to reject non-technician rows.
CREATE OR REPLACE FUNCTION fn_validate_technician_role()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM users
        WHERE id = NEW.technician_id AND role = 'technician'
    ) THEN
        RAISE EXCEPTION 'analytics_technician_daily: technician_id % is not a
technician', NEW.technician_id;

```

```

    END IF;
    RETURN NEW;
END;
$$;
CREATE TRIGGER trg_validate_technician_role
    BEFORE INSERT OR UPDATE ON analytics_technician_daily
    FOR EACH ROW EXECUTE FUNCTION fn_validate_technician_role();

CREATE TABLE analytics_business_daily (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    date              DATE                NOT NULL UNIQUE,
    total_jobs        INT                 NOT NULL DEFAULT 0,
    installations_completed INT           NOT NULL DEFAULT 0,
    complaints_resolved INT              NOT NULL DEFAULT 0,
    revenue            NUMERIC(14,2)     NOT NULL DEFAULT 0,
    pending_payment_total NUMERIC(14,2) NOT NULL DEFAULT 0,
    direct_jobs        INT                 NOT NULL DEFAULT 0,
    dealer_jobs        INT                 NOT NULL DEFAULT 0,
    repeat_complaint_count INT            NOT NULL DEFAULT 0,
    frequent_customer_count INT           NOT NULL DEFAULT 0,
    avg_resolution_mins NUMERIC(10,2),
    on_time_rate        NUMERIC(5,4),
    updated_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW()
);

CREATE TABLE analytics_brand_daily (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    brand_id          UUID                NOT NULL
        REFERENCES brands(id) ON DELETE RESTRICT,
    date              DATE                NOT NULL,
    installations_completed INT           NOT NULL DEFAULT 0,
    complaints_received INT              NOT NULL DEFAULT 0,
    complaints_resolved INT              NOT NULL DEFAULT 0,
    revenue            NUMERIC(14,2)     NOT NULL DEFAULT 0,
    revisit_count      INT                 NOT NULL DEFAULT 0,
    updated_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
    UNIQUE (brand_id, date)
);

CREATE TABLE analytics_dealer_daily (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id         UUID                NOT NULL
        REFERENCES dealers(id) ON DELETE RESTRICT,
    date              DATE                NOT NULL,
    jobs_submitted    INT                 NOT NULL DEFAULT 0,
    complaints_submitted INT            NOT NULL DEFAULT 0,
    installations_submitted INT          NOT NULL DEFAULT 0,
    resolved_count     INT                 NOT NULL DEFAULT 0,
    avg_resolution_mins NUMERIC(10,2),
    pending_count      INT                 NOT NULL DEFAULT 0,
    updated_at         TIMESTAMPTZ       NOT NULL DEFAULT NOW(),
    UNIQUE (dealer_id, date)
);

CREATE INDEX idx_tech_daily_date ON analytics_technician_daily (date);
CREATE INDEX idx_biz_daily_date ON analytics_business_daily (date);
CREATE INDEX idx_brand_daily_date ON analytics_brand_daily (brand_id,
date);

```

```
CREATE INDEX idx_dealer_daily_date ON analytics_dealer_daily (dealer_id,
date);
```

B.13 — Customer Reviews (Module 19)

```
CREATE TABLE customer_reviews (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id            UUID                NOT NULL UNIQUE
                        REFERENCES jobs(id) ON DELETE RESTRICT,
  technician_id     UUID                NOT NULL
                        REFERENCES users(id) ON DELETE RESTRICT,
  brand_id          UUID                NOT NULL
                        REFERENCES brands(id) ON DELETE RESTRICT,
  review_token      UUID                NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  link_generated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- Generated column enforces the 48-hour window (was unconstrained in v1).
  expires_at        TIMESTAMPTZ
      GENERATED ALWAYS AS (link_generated_at + INTERVAL '48 hours') STORED,
  submitted_at      TIMESTAMPTZ,
  star_rating       SMALLINT CHECK (star_rating BETWEEN 1 AND 5),
  is_low_rated      BOOLEAN
      GENERATED ALWAYS AS (star_rating <= 2) STORED,
  review_text       TEXT,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_reviews_technician ON customer_reviews (technician_id);
CREATE INDEX idx_reviews_low_rated
  ON customer_reviews (is_low_rated)
  WHERE is_low_rated = TRUE;
```

B.14 — State & Lifecycle Enforcement

```
-- jobs.status is now a proper enum (job_status), so only
-- enumerated values can ever be stored at the column level.
-- The trigger below enforces per-type validity as a second layer.

CREATE OR REPLACE FUNCTION validate_job_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
  valid_installation job_status[] := ARRAY[
    'pending_schedule', 'scheduled', 'assigned', 'acknowledged',
    'in_transit', 'in_process', 'completed', 'cancelled'
  ]::job_status[];
  valid_complaint job_status[] := ARRAY[
    'new', 'assigned', 'acknowledged', 'in_transit', 'in_process',
    'resolved', 'needs_revisit', 'revisit_scheduled', 'resolved_on_revisit',
    'cancellation_requested', 'cancelled'
  ]::job_status[];
BEGIN
  IF NEW.type = 'installation' AND NOT (NEW.status = ANY(valid_installation))
  THEN
    RAISE EXCEPTION 'Invalid installation status: %', NEW.status;
  END IF;
  IF NEW.type = 'complaint' AND NOT (NEW.status = ANY(valid_complaint)) THEN
```

```

        RAISE EXCEPTION 'Invalid complaint status: %', NEW.status;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validate_job_status
BEFORE INSERT OR UPDATE OF status ON jobs
FOR EACH ROW EXECUTE FUNCTION validate_job_status();

```

C. Relationships

From	To	Cardinality	Business rule / ON DELETE
virtual_customers	customer_phones	1 : N	ON DELETE RESTRICT — cannot remove VCID while phones exist
virtual_customers	jobs	1 : N	ON DELETE RESTRICT — cannot remove VCID while jobs exist
jobs	job_units	1 : N	ON DELETE CASCADE — units owned by job
jobs	job_assignments	1 : N (1 active)	ON DELETE RESTRICT — preserve assignment history
jobs	job_cancellation_requests	1 : 0..1	ON DELETE RESTRICT — UNIQUE enforces at most one request per job
jobs	revisits	1 : N	ON DELETE RESTRICT — preserve revisit history
revisits	revisit_assignments	1 : N (1 active)	ON DELETE RESTRICT
jobs	payments	1 : 1	ON DELETE RESTRICT — UNIQUE on payments.job_id
jobs	job_timeline	1 : N	ON DELETE RESTRICT — audit rows are immutable
jobs	notifications	1 : N	ON DELETE CASCADE — notifications follow their job
jobs	scheduling_conflicts	1 : N	ON DELETE RESTRICT
jobs	customer_reviews	1 : 0..1	ON DELETE RESTRICT — Module 19
users	job_assignments	1 : N	Technician assigned to many jobs
dealers	dealer_credentials	1 : 1	ON DELETE CASCADE — credentials owned by dealer
dealers	jobs	1 : N	ON DELETE RESTRICT
dealers	dealer_brands	N : M	via dealer_brands join table
brands	jobs	1 : N	ON DELETE RESTRICT
payment_methods	payments	1 : N	ON DELETE RESTRICT

D. Indexing Strategy

All indexes described in the v1 Section D table are now present in the DDL (they were listed but not created in the original schema).

Index target	Type	Justification
customer_phones.phone	B-tree	Phone-number lookup for VCID resolution on every new job
jobs(vcid, created_at DESC) WHERE !deleted	Partial composite	Repeat/frequent detection queries — most latency-sensitive path
jobs(status, type) WHERE !deleted	Partial composite	Dashboard job-list filtering
jobs(dealer_id, created_at DESC)	Composite	Dealer history view
jobs(brand_id)	B-tree	Brand analytics
jobs(scheduled_at)	B-tree	Scheduling conflict window checks
jobs(is_chronic) WHERE TRUE	Partial	Chronic-job dashboard queries
jobs(id) WHERE !deleted	Partial	Excludes soft-deleted rows from all operational queries
job_assignments(job_id) WHERE active	Partial unique	Enforces single active assignment; O(1) lookup
job_timeline(job_id, occurred_at DESC)	Composite	Ordered retrieval of all events per job
notifications(recipient_user_id, is_read, created_at DESC)	Composite	Badge count and unread list
notifications dedup (event_type, job_id, recipient_*)	Partial unique	Prevents duplicate notifications from retried workers
analytics_*_daily(date)	B-tree	Date-range dashboard queries
customer_reviews.is_lowRated WHERE TRUE	Partial	Owner dashboard low-rating filter

E. Concurrency & Data Integrity

E.1 — Optimistic Locking

Both jobs and payments use a version counter. The service layer reads the current version, passes it back on update, and raises HTTP 409 if the version has changed. The payment UPDATE now mirrors the jobs pattern (the guard was absent in v1).

```
-- Jobs update (unchanged from v1)
UPDATE jobs
SET    status      = :new_status,
       updated_at  = NOW(),
       version      = version + 1
WHERE  id          = :job_id
AND    version      = :expected_version
```

```

    AND is_deleted = FALSE;
-- rowcount = 0 → raise OptimisticLockException → HTTP 409

-- Payments update (version guard was missing in v1 – now enforced)
UPDATE payments
SET    amount           = :amount,
       payment_method_id = :method_id,
       last_edited_by   = :editor_id,
       last_edited_at   = NOW(),
       updated_at       = NOW(),
       version          = version + 1
WHERE  job_id = :job_id
       AND version = :expected_version;
-- rowcount = 0 → raise OptimisticLockException → HTTP 409

```

E.2 — Payment + Status Atomic Pair

```

BEGIN;
-- 1. Advance job status with version guard
UPDATE jobs
SET    status = 'completed', version = version + 1, updated_at = NOW()
WHERE  id = :job_id AND version = :v AND is_deleted = FALSE;

-- 2. Insert payment (UNIQUE on job_id prevents double-payment on retry)
INSERT INTO payments
(job_id, amount, payment_method_id, status, recorded_by)
VALUES
(:job_id, :amount, :method_id, 'collected', :technician_id);

-- 3. Append timeline entries
INSERT INTO job_timeline (job_id, event_type, actor_user_id, new_value)
VALUES (:job_id, 'status_transition', :technician_id,
        '{"status":"completed"}'::jsonb);

INSERT INTO job_timeline (job_id, event_type, actor_user_id, new_value)
VALUES (:job_id, 'payment_recorded', :technician_id,
        json_build_object('amount', :amount, 'method_id', :method_id)::jsonb);
COMMIT;
-- If jobs UPDATE returns 0 rows → ROLLBACK before attempting INSERT

```

E.3 — Analytics Idempotency

Analytics UPSERTs in v1 used additive increments with no deduplication, meaning a retried job event would double-count. The corrected pattern tracks which timeline event has been applied.

```

-- The analytics worker tracks which timeline event IDs it has processed.
-- On retry, it skips already-applied events rather than incrementing again.

-- Pattern: worker queries for unprocessed timeline events
SELECT jt.id, jt.event_type, jt.job_id, jt.occurred_at
FROM   job_timeline jt
WHERE  jt.event_type IN ('status_transition', 'payment_recorded')
       AND NOT EXISTS (
           SELECT 1 FROM analytics_processed_events ape

```

```

        WHERE ape.timeline_event_id = jt.id
    )
ORDER BY jt.occurred_at
LIMIT 500;

-- After UPSERT, mark the event as processed:
INSERT INTO analytics_processed_events (timeline_event_id, processed_at)
VALUES (:timeline_id, NOW())
ON CONFLICT (timeline_event_id) DO NOTHING;

-- Supporting table (add to migrations):
CREATE TABLE analytics_processed_events (
    timeline_event_id UUID PRIMARY KEY
    REFERENCES job_timeline(id) ON DELETE CASCADE,
    processed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

F. Improvements Summary

The table below maps every issue identified in the v1 review to its resolution in this corrected schema.

Severity	Issue	Fix applied
CRITICAL	No ON DELETE behaviour on any FK	Explicit ON DELETE RESTRICT / CASCADE / SET NULL on every FK throughout the schema
CRITICAL	jobs.status stored as TEXT	Replaced with job_status enum; trigger now operates on typed values
CRITICAL	customer_phones.linked_by not an FK	Added REFERENCES users(id) ON DELETE SET NULL
CRITICAL	payments version guard missing	UPDATE payments ... WHERE version = :expected_version now documented and enforced
HIGH	is_repeat / is_frequent stale after threshold change	Added repeat_window_days_used, frequent_threshold_used, frequent_window_used context columns
HIGH	is_chronic never auto-set	trg_chronic trigger on revisits INSERT sets jobs.is_chronic when sequence_number >= 3
HIGH	revisits.outcome was TEXT CHECK	Replaced with revisit_outcome enum
HIGH	scheduling_conflicts table missing from DDL	Full CREATE TABLE with FKs, unique constraint, and index added in B.10
HIGH	All 6 jobs indexes absent from DDL	All indexes from Section D now created in B.4
HIGH	payments.recorded_by cannot track office-staff edits	Added last_edited_by / last_edited_at columns with chk_edit_meta constraint
HIGH	customer_reviews.expires_at unconstrained	Now a GENERATED ALWAYS AS column (link_generated_at + 48h)

Severity	Issue	Fix applied
HIGH	analytics_technician_daily allows non-technician rows	trg_validate_technician_role trigger on INSERT/UPDATE
MEDIUM	PII spread / GDPR risk	Added repeat_window_days_used cols; GDPR note in Assumptions. Full PII extraction deferred per Assumption A-1.
MEDIUM	job_timeline actor columns both nullable with no guard	chk_actor constraint: at least one actor or event_type = 'system_event'
MEDIUM	Dealer password_hash in business table	Extracted to dealer_credentials table; supports OAuth/SSO extension without altering dealers
MEDIUM	visit_outcome enum name collides with column name	Enum renamed to arrival_outcome throughout
MEDIUM	system_config seed uses '<owner-uuid>' placeholder	Replaced with seed_system_config(p_owner_id UUID) migration function
MEDIUM	avg_star_rating 'Module 19 only' leaky abstraction	Column retained in base schema; behaviour controlled by system_config.customer_review_mode
RISK	Inconsistent soft-delete (is_deleted vs is_active)	is_deleted added to users, dealers, brands for uniform soft-delete semantics
RISK	Cancellation columns bloating jobs row	Extracted to job_cancellation_requests child table with ordering constraints
RISK	No deduplication on notifications	idx_notif_dedup_user and idx_notif_dedup_dealer partial unique indexes added
RISK	Analytics UPSERT double-counts on retry	analytics_processed_events table + idempotent worker pattern documented in E.3
RISK	Payment + status race on offline sync	Partial-commit risk documented; UNIQUE on payments.job_id is the backstop; pattern clarified in E.2

G. Assumptions

A-1 — Embedded PII retained by design

customer_name and address remain embedded in jobs rather than in a normalised customer_profiles table. This matches the stated v1 requirement. GDPR/right-to-erasure compliance must be handled at the application layer by nullifying these fields on verified erasure requests. A future migration to a separate profiles table is recommended when the user base exceeds ~50k VCIDs.

A-2 — Repeat/frequent flags are point-in-time

is_repeat and is_frequent are set at job-creation time and not retroactively updated when system_config thresholds change. The new context columns (repeat_window_days_used, etc.) record

which threshold was in effect, enabling accurate audit without retroactive flag migration. If retroactive recalculation is required, a background job should be run after each threshold change.

A-3 — dealer_credentials: one credential set per dealer

The dealer_credentials table uses a UNIQUE on dealer_id, enforcing one active credential set. Multi-device or token-based auth is out of scope for v2 but the separated table makes it straightforward to add an auth_tokens table later without altering the dealers row.

A-4 — analytics_processed_events is owned by the analytics worker

The analytics_processed_events table is managed exclusively by the NestJS analytics worker. No application business logic reads it. If the analytics schema is deployed separately, this table should live in that schema namespace.

A-5 — ON DELETE RESTRICT is the default policy

Unless noted otherwise, all FKs use ON DELETE RESTRICT. Physical deletion of any operational entity (user, dealer, brand, job) is intentionally blocked at the DB level. Deactivation via is_active = FALSE / is_deleted = TRUE is the correct operational path.